- `ActiveModel::Errors#generate_message` (which is used by Active Model validations but may also be used manually) uses `model_name.human` and `human_attribute_name` (see above). It also translates the error message and supports translations for inherited class names as explained above in "Error message scopes".
- `ActiveModel::Errors#full_messages` prepends the attribute name to the error message using a separator that will be looked up from errors.format (and which defaults to `"%{attribute} %{message}"`).

4.8.3 Active Support Methods

- `Array#to_sentence` uses format settings as given in the support.array scope.

5 How to Store your Custom Translations

The Simple backend shipped with Active Support allows you to store translations in both plain Ruby and YAML format.[2]
For example a Ruby Hash providing translations can look like this:

```
{
  pt: {
    foo: {
      bar: "baz"
    }
  }
}
```

The equivalent YAML file would look like this:

```
pt:
  foo:
    bar: baz
```

As you see, in both cases the top level key is the locale. `:foo` is a namespace key and `:bar` is the key for the translation "baz".
Here is a "real" example from the Active Support `en.yml` translations YAML file:

```
en:
  date:
    formats:
      default: "%Y-%m-%d"
      short: "%b %d"
      long: "%B %d, %Y"
```

So, all of the following equivalent lookups will return the `:short` date format `"%b %d"`:

```
I18n.t 'date.formats.short'
I18n.t 'formats.short', scope: :date
I18n.t :short, scope: 'date.formats'
I18n.t :short, scope: [:date, :formats]
```

Generally we recommend using YAML as a format for storing translations. There are cases, though, where you want to store Ruby lambdas as part of your locale data, e.g. for special date formats.

6 Customize your I18n Setup

6.1 Using Different Backends

For several reasons the Simple backend shipped with Active Support only does the "simplest thing that could possibly work" *for Ruby on Rails*[3] ... which means that it is only guaranteed to work for English and,